

신준섭 Fullstack Developer

nuclearbomb6518@gmail.com

소개

공기업을 다녔던 경험을 토대로, 일이 단순히 돈을 벌기 위한 수단이 아니라 삶의 원동력이 된다는 걸 깨달아 직장
의 안정성보다는 업무를 통한 성장과 성취감을 중요하게 생각합니다.

크고 복잡한 레거시 코드를 분석해 기능 개발, 문제 해결, 구조 개선을 하는 일을 오래 다뤘습니다. C++ 문서 편집기
와 웹오피스 공통 프레임워크부터 JSP→React 전환, 220개 테이블·약 200GB 규모의 MariaDB→PostgreSQL 이관,
Ncloud→Azure 이관을 거쳐, 지금은 5개 서비스가 한 서버에 얹혀 있던 IoT 플랫폼을 서비스별 분리 구조로 재설계
하고 있습니다. 전체 아키텍처를 정리해 팀에 배분한 뒤 프론트엔드 프레임워크, Go 이벤트 파이프라인, 빌드·배포
파이프라인, 모니터링까지 직접 맡고 있습니다.

맡은 일을 끝까지 책임지고, 스스로 계획을 세워 주도적으로 밀고 나갈 때 가장 잘합니다. 외주가 만들다 중단한 모
바일 앱을 인계받아 완성했고, 한 달째 멈춰 있던 아키텍처 논의를 정리해 전체 설계와 플랜을 세운 뒤 파트를 나눠
팀에 배분했습니다.

현 직장에서 맡은 일은 대부분 해본 적 없는 영역이었습니다. IoT 플랫폼도, Go도, 네이티브도, 배포 인프라도 그랬
습니다. AI를 그 간극을 메우는 수단으로 쓰며 일해왔고, 바쁠 때는 서너 개 작업을 동시에 돌립니다. 대신 생성된
코드를 그대로 올리지는 않습니다. 프레임워크를 건드리지 않는 화면·기능 추가는 코드를 확인하고 동작을 테스트
하는 선에서, 구조에 손이 닿는 작업은 이해될 때까지 설명을 요구한 뒤 반영합니다.

핵심 성과

- 차세대 IoT 플랫폼을 아키텍처 설계부터 구현까지 전 과정 담당 — 전체 구조를 정리해 팀에 배분하고, 프론트
엔드 프레임워크·Go 이벤트 파이프라인·빌드·배포 파이프라인·모니터링 네 축을 직접 구현
- 전국 편의점 프랜차이즈 약 1.8만 점포·일 약 5,200만 건의 센싱 데이터를 다루는 서비스를 운영·이관
- 220개 테이블·약 200GB 규모 MariaDB → PostgreSQL 전면 마이그레이션 (SQL dialect 변환 366개) — 이관 완
료 후 Ncloud 잔여 리소스 정리로 월 830만원 → 125만원
- Azure 리소스를 실사용량 기준으로 다운스케일·정리해 운영 환경 월 \$742 절감, Table Storage는 압축 아카이빙
파이프라인으로 월 123만원(증가 추세) → 48만원(고정) 전환
- FCM 푸시 처리 속도 90% 개선, 프로시저 쿼리 성능 95% 개선 (7분 → 20초)
- Python·Java·Node.js 3개 언어로 흩어진 배치를 공통 프레임워크 하나로 통합
- 운영 중인 모바일 앱 지표 (Google Play 기준) — 전국 편의점 프랜차이즈 앱 설치 8.16천·MAU 5.19천·비정상
종료율 0.02%, 근태관리 앱 설치 1.32천·MAU 1.19천·ANR 0.01%

경력

개발 경력 6년 6개월 (총 경력 10년)

주식회사 티앤애텍 대리 / 서비스팀

2025.04 ~ 재직 중 · 1년 4개월

- 전국 편의점 프랜차이즈 약 1.8만 점포를 포함해 5개 서비스(SEMS 편의점 / EMS / 근태관리 / 청소관리 / CCTV 전기차 화재 탐지)가 올라간 사내 IoT 플랫폼의 프론트엔드·백엔드·인프라 담당
- 점포당 약 10종의 센싱 데이터를 5분 주기로 수집 — 시간당 약 216만 건, 일 약 5,200만 건
- DB 구성 — PostgreSQL(220 테이블 · 약 200GB · pg_partman 일별 파티션) 중심에 MongoDB · Azure Table Storage 1차 적재 이중화, 디바이스 통신은 MQTT(EMQX)

SEMS 서비스 유지 관리 (2025.04 ~ 2026.05)

React, Next.js, TypeScript, JSP, Java, Spring, MariaDB, Android, iOS, FCM, Rollbar

- 서비스 규모 — 전국 편의점 프랜차이즈 약 1.8만 점포, 점포당 약 10종 센싱 데이터 5분 주기 수집 (시간당 약 216만 건 / 일 약 5,200만 건)
- Azure Table Storage → MongoDB → PostgreSQL로 이어지는 집계 서비스가 별도 Python 데몬으로 돌고 있던 것을 공통 배치 프레임워크로 이관해 통합 관리
- 모바일 웹 JSP → React 전환 인계 — 외주가 만들다 중단한 결과물의 미연결 기능과 잔존 오류를 순차 해결
- 푸시 오류 수정 및 FCM 최신화, 배치 처리로 로직 단순화 — 처리 속도 90% 개선
- Rollbar 모니터링 적용 — 모바일 프론트 오류 100% 제거, 모바일 WAS 오류 80% 제거
- 전력량 보정 로직 적용으로 80% 점포에서 한전 제공 실제 전력량과 오차 5% 이내 달성
- 서명, 이미지, 헤더 아이콘 등 공통 컴포넌트 구현 및 적용
- JSP/JS 파일 캐시 문제 쿼리스트링 처리, 공공데이터포털 사고 관련 기상청 API 오류 대응
- 기능 추가 — 간판 정기점검 타입·승인, 해피콜 이미지/장애유형 연동, 간판 AS 부품 수량, 양수도

SEMS 서비스 플랫폼 이관 (2025.06 ~ 2026.02)

PostgreSQL, MongoDB, Azure (Table Storage, Blob Storage, Service Bus, Function Apps), Spring Boot, Java, Python, MQTT, GitHub Actions

- 데이터 마이그레이션 — 220개 테이블 / 약 200GB / 전국 편의점 프랜차이즈 1.8만 점포 실시간 거래 시스템
 - MariaDB → PostgreSQL, Ncloud bucket storage → Azure Blob Storage
 - 이미지 SDK S3 → Azure Storage 전환 및 SAS token 적용
- SQL dialect 변환 366개 (MariaDB → PostgreSQL)
 - 쿼리 CTE 적용으로 가독성 개선
 - pg_stat_statements로 실행 통계 분석, 1초 이상 실행 쿼리 성능 개선
 - 중복 계산 제거, 인덱싱, pg_partman 테이블 파티셔닝
- 자바 배치 및 프로시저 이관
 - 성공/실패 로그 DB 저장, 에러 알림 슬랙 메시지, HikariPool·배치 처리 등 공통 로직 구현
 - 프로시저 쿼리 개선으로 약 7분 → 20초, 95% 성능 개선
 - 기상청(단기/초단기 예보, 특보), 환경공단(미세먼지), 알리고(카카오톡 알림톡) API 연동
- CI/CD 적용 — 수동 배포에서 GitHub Actions 자동화, pm2 + nginx 무중단 배포

- 데이터 집계 타임존 처리 — Table Storage·MongoDB는 UTC, PostgreSQL 집계 저장 시 비즈니스별 지정 타임존 적용
- 외부 연동 API 변경 협업(DR, 해피콜) — 요청/응답 상세 로깅, API 명세서 작성, CORS 허용 리스트 관리
- 스테이징 환경 도메인 및 SSL 적용, 매니저(설치) 앱 마이그레이션

TnM 플랫폼 위젯 개발 (2025.12 ~ 2026.01)

React, Next.js, TypeScript, react-flow

- 커스텀 차트 위젯 (실시간 데이터 시각화)
- 룰체인(react-flow) 노드 편집기 위젯
- 냉장비 정기점검 AI 대시보드 위젯 — 외부 AI 서비스가 산출한 이상 판정 결과를 조회·시각화 (모델 추론 자체는 AI 서비스 담당)

클라우드 환경 리소스 정리 및 비용 최적화 (2026.01 ~ 2026.07)

Azure, Azure Data Factory, Blob Storage, Ncloud, GitHub Actions

- Azure 리소스 비용 최적화 — 운영 환경 월 약 \$742 절감
 - 실사용량 대비 과한 성능의 리소스를 필요 성능 기준으로 다운스케일 (Service Bus, PostgreSQL server, MongoDB disk)
 - logAnalytics·Function Apps 설정 변경 및 VM 대체, 미사용 VM 제거
- Table Storage 아카이빙 이관 — 월 123만원(증가 추세) → 48만원(고정)
 - 최근 3개월치만 남기고 나머지는 Blob Storage에 압축 저장, Azure Data Factory 파이프라인으로 자동화
 - 일회성 마이그레이션 비용 92만원은 약 1.2개월 만에 회수
- Ncloud 잔여 리소스 정리 — 월 830만원 → 125만원 (플랫폼 이관에 따른 정리 작업)
 - 각 리소스의 사용 현황 파악 후 Azure로 이관, 미이관 서비스를 제외한 잔여 리소스 제거
 - VM·로드밸런서 정리, DB 가용성 해제, 컨테이너·VM 재배치
- GitHub self-hosted runner 적용 — 배포용 SSH 포트를 닫기 위한 1차 대응. 러너를 VM 안에 두어 외부 inbound 없이 배포되게 전환 (v2에서는 OIDC 무키 배포로 대체)

플랫폼 Task(배치) 구조 리팩토링 (2026.02)

Python, Java, Node.js, FastAPI, Uvicorn, systemd, pg_cron

- Python·Java·Node.js 3개 언어로 흩어진 배치를 하나의 운영 표준으로 통일 (로깅 포맷·Slack 알림 양식·설정 파일 구조)
- 7개 Python 서비스를 FastAPI + Uvicorn으로 통일, 14개 SQL Mapper 분리
- Java 배치 10여 개·PostgreSQL 프로시저 10여 개를 공통 베이스/단일 진입 프로시저 패턴으로 통합 — 실행 로그·예외·Slack 알림·설정 로딩을 베이스가 처리하고 자식은 비즈니스 로직만 작성
- Table Storage → MongoDB → PostgreSQL 집계 서비스를 배치 프레임워크로 이관 — 단일 파이썬 데몬이 메모리 스케줄러로 돌리던 수집 작업 12개 → 8개 systemd 배치로 재편 (5분 3 / 1시간 3 / 1일 2)
- pg_cron 스케줄 등록, pg_partman 파티셔닝, pg_notify 기반 Slack 에러 알림 파이프라인 적용
- 폴더 구조 개선, graceful shutdown 및 Azure Monitor 적용

모바일 네이티브 개선 (2026.04 ~ 2026.05)

React, Next.js, TypeScript, Android, iOS, FCM

- id/pw를 기기 localStorage에 평문 저장하던 자동로그인 방식 개선
- url param·쿠키·localStorage에 의존하던 네이티브↔웹앱 데이터 전달(deviceType, osVersion, pushToken) 방식 정비
- Android 카메라·이미지 권한 정책 변경 적용 — 권한 요청 로직을 공통 컴포넌트로 구현해 흩어진 사용처를 일괄 처리
- 앱링크 적용으로 푸시 클릭 시 targetUrl 페이지 이동, 푸시 클릭 후 알림이 리스트에 남던 오류 해결
- 푸시 토큰 만료 시 자동 재발급 (기존에는 앱 재설치 필요), FCM 전송 후 미수신 추적을 위한 푸시 로그 적재
- 앱 버전 체크 후 강제 업데이트 다이얼로그 및 스토어 이동 도입, 모바일 앱 전용 테스트 환경 구축
- 운영 지표 (Google Play, 최근 28일) — 전국 편의점 프랜차이즈 앱 설치 8.16천 · MAU 5.19천 · 비정상 종료율 0.02%(직전 28일 대비 -0.02%p), 근태관리 앱 설치 1.32천 · MAU 1.19천 · ANR 0.01%

IoT 플랫폼 v2 재설계 (2026.06 ~ 진행 중)

Next.js, TypeScript, Go, Java, Spring Boot, Kafka, Azure Event Hubs, PostgreSQL, MongoDB, EMQX, Docker, GitHub Actions, Terraform, ClickHouse, OpenTelemetry

- 한 달째 멈춰 있던 아키텍처 논의를 정리해 전체 설계와 플랜을 수립하고, 파트를 나눠 팀에 배분. 이후 프론트엔드·이벤트 파이프라인·빌드/배포·모니터링 네 축을 직접 구현
- 5개 서비스가 한 서버에 얹혀 있던 구조를 서비스별 분리 구조로 전면 재설계 — DB는 스키마, API 서버는 컨테이너 단위로 분리, 프론트는 웹/모바일만 분리하고 앱 간 코드 공유
- 수집 경로 Function Apps → Kafka 전환 — Event Hubs 파티션 4 · 보존 7일 · Capture(Avro, 60초/10MB) 아카이빙으로 유실 없는 수집, consumer lag 기반 오토스케일(KEDA, replica ≤ 4)
- 프론트엔드 프레임워크 직접 담당 — 편집/뷰 모드, 위젯·레이아웃 배치, 전역 상태 설계, API 레이어(공통 클라이언트·인증·에러 처리·서버 상태 캐싱), 디자인 시스템, 다국어(ko/en) 리소스 구조 및 비즈니스별 타임존 정책
- 공통 커널 + 앱 플러그인 구조 — 신규 앱 온보딩을 매니페스트 선언 + DB 구성으로 좁혀 커널 무변경. 리전/페이지바디/위젯을 레지스트리로 해석하고, 스타일은 토큰 계약(코드) + 값(DB) cascade로 분리. 플러그인 정의 DB화 3단계를 마쳐 매니페스트 합성이 DB의 i18n·style·region 구성까지 소비
- 레거시 5개 앱을 3비즈니스 5플러그인으로 이식하고 목업을 실 API로 전량 배선 — 공통 도메인·감사 로그(CSV 서버 내보내기)·시설관리·청소관리·근태관리. 앱 전용 서브도메인 기반 테넌트 호스트 라우팅과 응답 스키마 캡 리포트 적용
- 관리자·비즈니스 기본앱(웹·모바일) 제공 기능 기획 — 6개 서비스 명세 유스케이스 64건을 표면별로 배정하고 화면 필요 50건을 선별. 화면·위젯을 먼저 만들어 필요한 API 계약을 캡 문서로 정리해 백엔드에 요청하는 방식으로 진행
- 모바일 셸 — Expo(React Native) WebView 셸과 웹↔네이티브 브리지(push·camera·qr 등 feature 선택) 구조 설계. 화면은 웹으로 렌더하고 네이티브는 셸·브리지 역할만 맡는 래핑 방식
- Go 이벤트 파이프라인 직접 담당 — 수신 어댑터 → 판단 코어 → 실행기 3단 구조, 먹등 3중(스토어 UNIQUE · dispatcher 중복 skip · executor dedup) + DLQ로 유실·중복 방지. 비즈니스별 파이프라인 세트로 재구조화해 비즈니스당 프로세스 6개 → 2개로 축소
- 게이트웨이 협의의 프로토콜을 캡 카탈로그로 전량 가시화한 뒤 완주, 요청·응답 감사로 거절과 미도달을 구분. AI 추론 연동 프로세서는 외부 추론 서비스 호출 어댑터로 요청 매핑·응답 정규화를 맡고 이상 감지 시에만 제어 프로세서로 체이닝 (실 AI 서비스 연결은 진행 중)

- Go 선택 근거 — 가벼운 동시성으로 GIL 우회(멀티프로세스 + VMSS)가 불필요해지고, 정적 컴파일 단일 바이너리로 컨테이너 이미지가 수백 MB → 수십 MB
- 빌드·배포 파이프라인 구축(담당 공백으로 직접 인수) — v1에서 SSH 포트 피격 대응으로 도입했던 self-hosted runner를 v2에서는 Azure Entra ID + OIDC 무키 배포로 대체. 상시 점유 러너와 보관이 필요한 키를 함께 제거하고 빌드 속도도 회복. 워크플로우·secret을 별도 레포로 통합(Key Vault 이관 예정)하고 Terraform은 인스턴스 관리·관리형 서비스로 한정 적용
- pm2·jar·Docker 혼재 실행을 docker compose로 통일 — 배포 스크립트를 서비스 수만쯤에서 1개로 축소. 프론트·공통 API·모니터링·EMQX·MongoDB·APIM 적용 완료, 파이프라인·잔여 API 서버 진행 예정. 빌드·배포를 분리해 기빌드 커밋 재배포는 수 초
- 모니터링 구축 — 헬스체크·로그·리소스 상태값 셋을 저장소를 나누지 않고 ClickHouse 한 곳에 수집(VM 3대·서비스 12종). Prometheus·Loki·트레이싱 백엔드로 삼분되는 구성을 단일 서비스로 대체, 보존은 로그 14일·트레이스 7일·지표 30일 차등
- OpenTelemetry Collector 2계층 — 에이전트는 VM 로컬만 수집하고 게이트웨이가 유일한 적재 지점이라 자격증명이 한 곳에 모임. 조회 API를 공통 API 모듈로 넣고 관리자 앱에 모니터링 메뉴와 상태·로그·리소스 3화면 구성, 개발 환경 배포·연동 완료

팀 협업 체계 수립 (2025.09 ~)

- 비즈니스별 슬랙 모니터링/유지보수 채널 생성 및 공유
- 지라 팀 스페이스 생성 및 업무 관리 — 전체 스프린트와 동일 스토리 관리, 하위 항목은 담당자가 자유롭게 추가, 기한에 따른 중요도 변경 자동화
- GitHub 브랜치 관리 룰 적용

주식회사 엑스 프론트엔드 개발자

2023.09 ~ 2023.12 · 4개월

React, JavaScript, TypeScript, Styled-Components, SWR, Redux

제품 안정화

- 백로그 포함 할당 이슈 약 180개 중 78%, 140개 해결
- 백로그 제외 전체 해결

신기능 구현

- 계정관리 — 계정의 계좌/채널/계약 정보 관련 생성/상세/수정 페이지 추가
- 상품 판매단위 생성 — 상세/수정 페이지 추가
- 상품 미리보기 — 가채널 판매 화면 및 비회원 공유용 미리보기 화면 제공

CI/CD 및 공통 구조

- TypeScript, ESLint, Prettier 적용 (새 구현부부터)
- Git hooks 추가 — pre-commit 시 type check 및 lint check, PR action으로 패키지 설치 및 빌드 수행
- 디자인 시스템 구축 — Color, Label, Icon, InputField, TextArea, Dropdown, Dialog, Modal 등
- 상태 관리 라이브러리 — 기존 ContextAPI·react-hook-form·zustand에서 Redux, SWR 적용
- 날짜 관리 — moment → date-fns 교체

티맥스가이아 (구 티맥스에이앤씨) C++ / 프론트엔드 개발자

2020.02 ~ 2023.09 · 3년 7개월

2020.08 사명 변경 — 동일 회사에서 연속 재직

ToHangul 한글 문서 편집기 개발 (2020.02 ~ 2022.02)

C++, OOXML, MVC

- 쪽 번호 매기기 — 동작 정상화 및 오피스 소기능 페인트 구조 파악
- 머리말꼬리말 — 오피스 레이아웃 구조 파악, 바이너리 스펙 read/write 구현, 편집/템플릿 기능 구현
- 확대/축소, 인쇄/미리보기/회색조 — 오피스 뷰컨트롤러/페인트 구조 파악, 편집기호 등 편집화면과 다른 인쇄/미리보기 화면 전체처리(paint vs print), 화면 스케일링·필터링 처리
- 줄번호 — 기존 event-driven 구조를 이용한 카운팅 및 페인트 기능 구현
- KERIS(한국교육학술정보원) · 서울시 교육청에 ToHangul 서비스 제공

웹오피스 클립보드 프레임워크 (2022.02 ~ 2023.09)

React, TypeScript, MobX

- 오피스 클립보드 포맷 데이터를 Object화해 sync 보장 및 큐처럼 관리하도록 리팩토링
- 표준 웹 클립보드 API를 이용한 외부 오피스 데이터 호환 프레임워크 구현
- 데이터 포맷 간 변환을 위한 parse/write 구조 구현
- copy, paste 관련 pre/post 프로세스 구조 개선

웹오피스 셀렉션 프레임워크 (2022.06 ~ 2023.09)

React, TypeScript, MobX

- 블럭 셀렉션 처리를 위해 모델 정보를 이용해 셀렉션을 자동 생성하도록 리팩토링
- 도형·표 등 오피스 자체 셀렉션 및 공동 편집 시 세션별 처리
- 중복이나 불필요한 구조 개선 및 전처리/후처리 로직 추가

웹오피스 클라이언트 **CI/CD** (2023.02 ~ 2023.09)

Webpack, Jest, React, GitLab CI

- 빌드 성능 개선 — webpack v4 → v5, babel-loader/terser-webpack-plugin → esbuild-loader/esbuildMinifyPlugin
- 테스트 성능 개선 — jest v26 → v29, 비동기 로직(리소스 로딩 등) mocking
- React v16 → v18 마이그레이션 및 관련 패키지 전체 버전 업데이트
- GitLab runner docker 메모리 제한으로 jest 실행 중 메모리 충돌 파이프라인 실패 개선
- husky Python → Node.js 마이그레이션, MR action에서 cp를 mv로 변경해 속도 개선
- 빌드 환경별 옵션 및 패키지 관리, 배포 패키징

한국수력원자력(주) 발전소 현장근무

2016.08 ~ 2020.01 · 3년 6개월

- 원자력발전소 발전 교대 근무
- RI(방사선동위원소취급자 일반면허) 자격증 취득

학력

서울대학교 학사 / 원자핵공학과

2010.03 ~ 2016.08 · 졸업

- 상압 플라즈마(DBD)의 자기장 분포 측정 및 분석

스킬

Frontend React, Next.js, TypeScript, JavaScript, TailwindCSS, TanStack Query, Zustand, Radix UI, MobX, Redux, SWR, Styled-Components

Backend Java, Spring Boot, Python, FastAPI, Node.js, Go, C++

Data PostgreSQL, MariaDB, MongoDB, Azure Table Storage, SQL

Infra Azure, Ncloud, Docker, Docker Compose, Terraform, GitHub Actions, pm2, nginx, systemd, EMQX/MQTT, Kafka, Azure Event Hubs, WebSocket

Observability ClickHouse, OpenTelemetry, Azure Monitor, Rollbar

Mobile Android, iOS, FCM, React Native/Expo (WebView 셀 · 브리지)

AI 활용

도구 · 모델

- Cursor → Antigravity → tmux 기반 CLI → orca 순으로 작업 환경을 옮겨오며 정착
- Claude를 주력 모델로 쓰고, Codex와 Antigravity(Gemini)를 용도에 따라 병행
- 사내 기획 문서는 Visily(비즐리)로 작성된 화면 설계를 전달받아 개발에 착수

활용 방식

- Superpowers 하네스를 주로 사용 — 브레인스토밍으로 요구사항을 먼저 정제하고, 계획 수립 → TDD → 코드 리뷰 → 완료 전 검증으로 이어지는 프로세스를 강제. 근본 원인을 먼저 찾는 4단계 디버깅 절차도 함께 사용
- Codex는 주로 계획 검토와 PR 리뷰 용도로 활용 — 다른 모델의 시선으로 한 번 더 걸러내는 역할
- MCP 서버·서브에이전트를 연동하고, CLAUDE.md 행동 규칙과 커스텀 스킬을 직접 작성해 작업 환경을 구성
- 플랫폼 v2 프론트엔드 디자인 시스템을 Claude의 디자인 기능으로 구성하고, 그 결과를 디자인 문서로 정리해 구현에 반영
- AI가 코드를 망치는 전형적인 패턴(추측으로 진행, 불필요한 추상화, 요청 범위 밖 수정)을 막기 위해 행동 원칙을 문서로 정의해 적용 — 가정 명시, 최소 구현, 변경 범위 최소화, 검증 가능한 목표화

제품 적용

- 서비 챗봇 — OpenAI API 기반 대화 컨텍스트 유지 및 WebSocket 스트리밍 응답
- TechFeed — Gemini 2.5 Flash로 콘텐츠 요약, Redis 7일 캐시로 중복 호출 및 토큰 비용 절감
- 온톨로지·GraphRAG — 제조 공정 데이터 적용과 개인 프로젝트 반영을 목표로 학습 진행 중

활용 관점

- 익숙하지 않은 언어를 다루는 부담이 크게 감소 — Go 이벤트 파이프라인, iOS Objective-C, Android Java 등 주력이 아니던 영역의 작업도 직접 담당 가능
- 맡을 수 있는 업무 범위와 처리량이 함께 증가 — 바쁠 때는 작업 3~4개를 동시에 진행. 업무 외에도 개인 프로젝트, 기술 학습, 자격증 준비, 일정 관리에 같은 방식으로 활용
- 리뷰 깊이는 프레임워크를 건드리는지로 나눔 — 구조에 손이 닿지 않는 화면·기능 추가는 코드 확인과 화면 상태·기능 동작 테스트로 갈음하고, 구조를 바꾸는 작업은 이해되지 않는 부분의 설명을 요구해 납득한 뒤 반영. 모르는 코드를 그대로 올리지 않는다는 선은 작업 성격과 무관하게 유지
- 생성된 코드가 동작하더라도 프로젝트 컨벤션에서 벗어나는 경우가 있어, 구조가 걸린 작업일수록 리뷰 비중을 높게 가져가는 편

자격증

- 정보처리기사
- AZ-900 (Microsoft Azure Fundamentals)
- SQLD (SQL 개발자)
- ADsP (데이터분석 준전문가)
- RI (방사선동위원소취급자 일반면허)

포트폴리오 · 기타 문서

- 포트폴리오 — <https://profile.nuclearbomb6518.com>
- GitHub — <https://github.com/Junseop-Shin>
- 업무 기록 — <https://github.com/Junseop-Shin/Work>